



CurriculumTutor: An Adaptive Algorithm for Mastering a Curriculum

K. M. Shabana¹(✉), Chandrashekar Lakshminarayanan², and Jude K. Anil¹

¹ Indian Institute of Technology Palakkad, Palakkad, India
shabana.meethian@gmail.com

² Indian Institute of Technology Madras, Chennai, India
chandrashekar@cse.iitm.ac.in

Abstract. An important problem in an intelligent tutoring system (ITS) is that of adaptive sequencing of learning activities in a personalised manner so as to improve learning gains. In this paper, we consider intelligent tutoring in the *learning by doing* (LbD) setting, wherein the *concepts* to be learned along with their inter-dependencies are available as a *curriculum graph*, and a given concept is learned by performing an activity related to that concept (such as solving/answering a problem/question). For this setting, recent works have proposed algorithms based on multi-armed bandits (MAB), where activities are adaptively sequenced using the student response to those activities as a direct feedback. In this paper, we propose CurriculumTutor, a novel technique that combines a MAB algorithm and a *change point detection* algorithm for the problem of adaptive activity sequencing. Our algorithm improves upon prior MAB algorithms for the LbD setting by (i) providing better learning gains, and (ii) reducing hyper-parameters thereby improving personalisation. We show that our tutoring algorithm significantly outperforms prior approaches in the benchmark domain of two operand addition up to a maximum of four digits.

Keywords: Adaptive sequencing · Multi-armed bandits · Change-point detection · Personalisation

1 Introduction

Personalised learning approaches tailored to address the individual needs, skills, and interests of each student, have been found to cause a significant improvement in learning gains for the students, apart from providing an engaging learning experience. Intelligent Tutoring Systems (ITS) have been effective in delivering personalised learning to students in an automatic manner. An ITS consists of three important components namely: (i) *domain model* that captures the relations or dependencies between the various concepts to be learned, (ii) *student model* that represents the student's current knowledge level and how it changes as the tutoring progresses and (iii) *tutoring model* that decides the sequence of

learning activities presented to students. The tutoring model performs *adaptive sequencing of activities*, wherein learning activities to be presented to students are selected based on student competence levels estimated through their interactions with the system. This leads to better personalisation of learning and hence higher learning gains.

Clement et al. [4] proposed a tutoring algorithm called ZPDES that uses a *curriculum graph* capturing the concept inter-dependencies as a domain model. The key highlight of ZPDES is the use of ideas from multi-armed bandits to perform adaptive assignment of learning activities based on student responses. Later Brunskill et al. [10], used ZPDES to propose an adaptive sequence of activities to advance students through a curriculum graph generated based on an algorithmic representation of the concepts for the domain of two operand addition upto a maximum of 4 digits.

Our Contribution: In this paper, we consider the *learning by doing* (LbD) setting used in [10], wherein the concepts to be mastered are available in the form of a curriculum graph and the students learn a concept by performing activities related to the concept. Even though ZPDES is less reliant on the underlying student learning model, it has its own set of hyper-parameters which adversely affect personalisation. Our main contribution is a novel tutoring algorithm called CurriculumTutor for the LbD setting. CurriculumTutor combines the ideas of multi-armed bandits and *change point detection*. Here, the change point detector separately tracks the mastery level of each activity and an upper confidence based MAB algorithm manages the exploration vs exploitation tradeoff. We show that our CurriculumTutor significantly outperforms ZPDES in the benchmark domain of two operand addition up to a maximum of four digits, especially in scenarios where the learner faces difficulty in learning a subset of concepts in the curriculum graph.

2 Background

In this section, we first describe the learning by doing setting and then explain how prior works have used ideas from multi-armed bandits in the tutoring model.

Learning by Doing (Fig. 1): A *concept* is a representational unit of educational content which is learned by performing an activity related to that concept (such as solving/answering a problem/question). A *concept/curriculum graph* has concepts as nodes and the edges represent dependency/prerequisite relations between concepts. Several methods have been proposed in the literature to construct concept graphs from text corpus [6, 13]. In this paper, we assume access to a concept graph for the curriculum to be mastered. *Zone of Proximal Development (ZPD)* is a set of concepts on the boundary of the student’s knowledge (Fig. 1). It is based on an idea from classical psychology and education research which states that learning is the fastest and most engaging when practicing on material slightly beyond the current abilities of the student [3]. A *Question bank*

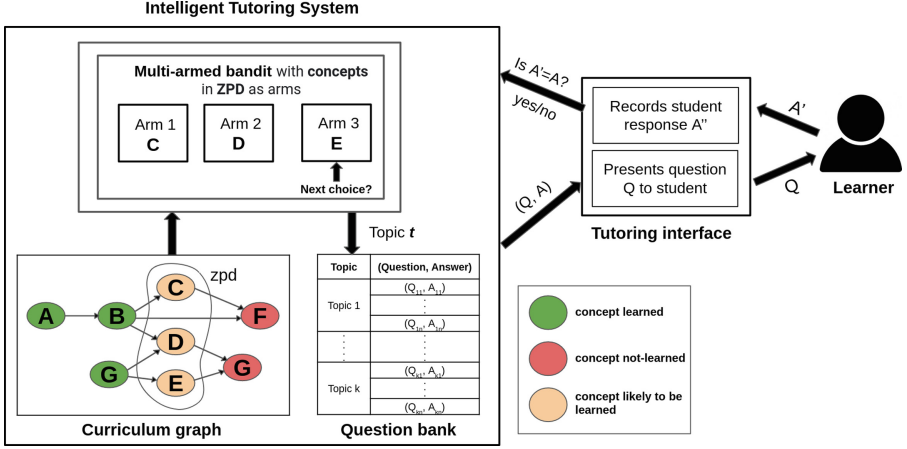


Fig. 1. Multi-armed bandit framework for adaptive activity sequencing in an ITS

contains a collection of question answer pairs associated with each concept. It is further assumed that all the questions associated with a concept are of the same difficulty level.

Multi-armed bandit (MAB) problem consists of a fixed set of actions (arms), and an unknown reward distribution associated with each arm. The system has to select actions (arms) such that its long term cumulative reward is maximized. The key challenge in this problem is to balance exploiting the information that has already been gained about the effectiveness of each action and exploring actions where the estimates about their value are still relatively uncertain, referred to as the *exploration-exploitation* tradeoff. Over time the system learns which actions are more effective and can earn larger rewards.

MAB Applied to Tutoring Model: In the multi-armed bandit framework for adaptive sequencing, the activities/lessons in the ZPD form the arms and rewards are computed based on student responses, as seen in Fig. 1. The system maintains an estimate of the learning progress associated with each concept in the ZPD, based on which it decides the next activity to be offered to the student. In the process of maximizing cumulative rewards, the MAB framework would pick activities from the concepts that the student tends to learn faster, thereby generating a personalized activity sequence. This formulation is less reliant on the underlying model of student learning and can therefore better adapt to different individual learning behaviors.

Algorithm 1. ZPDES

Initialization: For each activity a in ZPD in the **not-learned** state set $w_a = w_0$

Selecting the next activity: The weights (w_a) of the activities in the ZPD are normalized (w'_a) and an activity is sampled from the probability distribution p_a

$$w'_a = \frac{w_a}{\sum_{a' \in ZPD} w_{a'}}, \quad p_a = w'_a(1 - \gamma) + \gamma \frac{1}{|ZPD|} \quad a \in ZPD$$

Reward for the chosen activity a' : $r = \sum_{k=n_{a'}-d/2+1}^{n_{a'}} \frac{C_{a',k}}{d/2} - \sum_{k=n_{a'}-d}^{n_{a'}-d/2} \frac{C_{a',k}}{d/2}$
 n_a is the number of times activity a has been chosen

Updating weight of activity a' : $w_{a'} \leftarrow \beta w_{a'} + \eta r$

3 Prior Work and Research Gaps

In this section, we discuss the ZPDES algorithm proposed by Clement et al. [4] and then look at the research gaps in ZPDES, in particular about how the hyper-parameters in ZPDES adversely affect personalization.

Zone of Proximal Development and Empirical Success (ZPDES) [4] (see Algorithm 1) is based on the multi-armed bandits framework for adaptive activity sequencing where pre-conditions between activities are provided in the form of an expert defined curriculum graph. At each time-step, ZPDES selects the next activity a' based on the normalized weights of activities (w'_a) in the ZPD as well as an exploration factor γ , $0 \leq \gamma \leq 1$. The correctness of student response to the activity a' at the i^{th} attempt, $C_{a',i}$, is a binary random variable - it takes a value of 1 for a correct response and 0 for incorrect response. The reward is computed by taking the difference of the number of successes in the last $d/2$ samples with the $d/2$ previous samples, where d is the window-size. The reward provides an empirical measure of how the success rate is increasing. The reward becomes close to zero when either a concept has been mastered or the student struggles to master it. A high positive reward indicates good learning progress, whereas a negative reward implies stalled learning. These rewards are then used to update the activity weights ($w_{a'}$) based on the hyper-parameters η and β . ZPDES doesn't propose a method to determine if a concept has been mastered or not.

Later Brunskill et al. [10] proposed a novel system that combines automatic curriculum ordering [1] with ZPDES for automatically and adaptively advancing a student through a curriculum. A 'sliding windowed' average was used in this framework for inferring mastery, where a concept is determined to be *mastered* when the accuracy over the past d attempts reaches above a specified threshold t .

Research Gap: The major drawback of ZPDES is the use of many hyper-parameters such as d, w_0, γ, β and η , which are critical for its deployment, and eventual success. Using fixed values for these hyper-parameters would have an adverse effect on personalisation since the same set of hyper-parameters may

not be effective for the entire population. On the contrary, tuning these hyper-parameters for different individuals is an additional computational overhead and a time consuming process.

4 Our Approach: Breaking Adaptive Activity Sequencing into Sub-problems

Our algorithm **CurriculumTutor** addresses the research gap in the previous section. We describe our approach in this section and present the algorithm in the next section. The main reason for the presence of hyper-parameters in ZPDES is that it tries to address together both the exploration vs exploitation tradeoff as well as the non-stationarity of the student responses. On the contrary, our approach is to decompose the adaptive activity sequencing problem into the following two sub-problems namely (i) **activity selection** to take care of the exploration vs exploitation tradeoff, and (ii) **detection of mastery** to take care of the non-stationarity of the student responses. We now describe these in detail.

4.1 Activity Selection

The tutoring algorithm at each time step has to select an activity that offers higher learning progress to the student. In the multi-armed bandit framework for activity selection, the concepts in the ZPD form the arms while rewards are computed based on student responses. We consider the case where the rewards are binary, i.e., a reward of one is obtained if an activity related to the selected concept is correctly performed and otherwise zero. The problem can now be modelled as a *Bernoulli multi-armed bandit*, where a reward of 1 is obtained with probability p that is dependent on the knowledge level of the associated concept(arm).

4.2 Detection of Mastery

The tutoring algorithm has to infer whether a concept has been mastered based on the sequence of student responses. Determining whether a concept has been mastered helps to avoid over-practicing which leads to a drop in the learning gains, and under-practicing that results in the concept not being learned.

Each concept is assumed to be associated with two states - *learned* and *not-learned*. Let r_i denote the correctness of the student response at the i^{th} attempt. When the concept has not been mastered, $r_i \sim P_{\text{not-learned}}$, where

$$P_{\text{not-learned}}(r = 1) = p_{\text{guess}} \quad P_{\text{not-learned}}(r = 0) = 1 - p_{\text{guess}} \quad (1)$$

When the concept has been mastered $r_i \sim P_{\text{learned}}$, where

$$P_{\text{learned}}(r = 1) = 1 - p_{\text{slip}} \quad P_{\text{not-learned}}(r = 0) = p_{\text{slip}} \quad (2)$$

The reward distribution is now *piecewise stationary* i.e., remains constant for a certain period, and shifts at some unknown time step referred to as a change-point, that corresponds to the transition from the *not-learned* to *learned* state. Detection of mastery of a concept can now be modelled as a *change-point detection problem*, which can be described as follows [12]: Given a sequence of observations $\{X_n\}_{n \geq 1}$, the observed random variables $X_1, X_2, \dots$ have distribution function p_0 until a change occurs at an unknown point in time $\lambda, \lambda \in \{1, 2, \dots\}$, after which the observations have another distribution $p_1, p_0 \neq p_1$. A sequential change-point detection procedure is identified with a stopping time τ for an observed sequence $\{X_n\}_{n \geq 1}$, i.e., the time of alarm τ at which it is declared that a change has occurred, which is a random variable depending on the observations. The *average detection delay (ADD)* and *false alarm rate (FAR)* of a detection procedure is defined as follows:

$$ADD_\lambda(\tau) = E_\lambda(\tau - \lambda | \tau \geq \lambda) \quad (3)$$

$$FAR(\tau) = \frac{1}{E_0(\tau)} \quad (4)$$

where $E_0(\tau)$ denotes the expectation of the sequence $\{X_n\}_{n \geq 1}$ when there is no change, i.e., $\lambda = \infty$. A good detection procedure should have small values of average detection delay with a low FAR.

5 Our Algorithm: CurriculumTutor

We now describe CurriculumTutor, a novel tutoring algorithm that combines a Bernoulli multi-armed bandit algorithm with a change point detection technique for adaptive activity sequencing to master a curriculum. The algorithm maintains a zpd at each time step, based on which activities are presented to the student. A concept is removed from the ZPD when it is *determined* to have been transitioned into the learned state. The ZPD is then updated with concepts in the not-learned state, all of whose prerequisites are in the learned state. This continues until the student has mastered all the concepts in the curriculum graph. The *CurriculumTutor* algorithm has been described in Algorithm 2.

CurriculumTutor uses the *Kullback-Leibler Upper Confidence Bound (KL-UCB)* algorithm to perform activity selection, as it is shown to have better performance bounds than UCB and its variants [8]. The change-point detection algorithm used by CurriculumTutor is *Cumulative Sum (CUSUM)*. Here the log-likelihood ratio (LLR) is used to test the hypothesis that a change occurred at the point λ and that there is no change at all ($\lambda = \infty$) which is defined as:

$$Z_{n,\lambda} = \sum_{k=\lambda}^n \log \frac{p_1(X_k | X_1, \dots, X_{k-1})}{p_0(X_k | X_1, \dots, X_{k-1})}, \quad n \geq \lambda \quad (5)$$

Here, p_0 and p_1 are the pre-change and post-change probability density functions respectively. The maximum LLR statistic $U_n = \max_{1 \leq \lambda \leq n} Z_{n,\lambda}$ is compared with

Algorithm 2. CurriculumTutor

```

1: Initialize  $zpd = \text{Minimal elements of the partial ordering curriculum, } P$ 
2: Add arms corresponding to elements in  $zpd$ 
3:  $t = 1$ 
4: repeat
5:    $c_t = \text{SELECT-ACTIVITY}(zpd)$ 
6:   Present an activity of type  $c_t$  and observe correctness of response  $r_t$ 
7:   Update estimates associated with  $c_t$  based on  $r_t$ 
8:    $\text{DETECT-MASTERY}(c_t)$ 
9:    $t \leftarrow t + 1$ 
10: until  $zpd$  is empty
11: procedure  $\text{SELECT-ACTIVITY}(zpd)$ 
12:   return concept selected by KL-UCB
13: end procedure
14: procedure  $\text{DETECT-MASTERY}(c)$ 
15:   Update CUSUM estimate for  $c$ 
16:   if change-point detected for  $c$  then
17:     Mark  $c$  as learned
18:     Remove  $c$  from  $zpd$ 
19:      $zpd \leftarrow zpd \cup \{q' \in P \mid (c \leq q') \wedge (\nexists p \in P \text{ such that } p \leq q' \wedge p \text{ is not-learned})\}$ 
20:     Add arms corresponding to new elements in  $zpd$ 
21:   end if
22: end procedure

```

a threshold h and a change is detected when the value of U_n exceeds h . When $h > 0$ and the observations are independent and identically distributed (i.i.d.), U_n can be replaced by the statistic $\tilde{U}_n$ which obeys the recursion:

$$\tilde{U}_n = \max \left\{ 0, \tilde{U}_{n-1} + \log \frac{p_1(X_n)}{p_0(X_n)} \right\} \quad (6)$$

with the initial condition $\tilde{U}_0 = 0$. It has been proved [9] that in the i.i.d. case CUSUM minimizes the worst case average detection delay among all the detection algorithms for which the FAR is fixed at a given level $\overline{FAR}$. With p_{guess} and p_{slip} defined by the user, the pre and post change distributions are as follows:

$$p_0(k) = p_{guess}^k (1 - p_{guess})^{(1-k)} \quad (7)$$

$$p_1(k) = (1 - p_{slip})^k (p_{slip})^{(1-k)} \quad \text{for } k \in \{0, 1\} \quad (8)$$

CurriculumTutor uses the $\tilde{U}_n$ statistic for change-point detection, as the correctness of student responses are independent random variables given the state of each concept.

6.2 Student Models for Simulation

Students were simulated using the following popular knowledge tracing models. For each student model, separate knowledge components were defined corresponding to addition of different lengths, presence of carry and overflow. Prerequisites were imposed on knowledge components and each problem is associated with exactly one knowledge component. All activities are assumed to be of the same difficulty level.

Bayesian Knowledge Tracing (BKT). Here a student's knowledge state is modelled as a two-state Hidden Markov Model (HMM) [5], one per knowledge component/skill, where the skill is either mastered by the student or not. Prerequisites between knowledge components were enforced by varying the transition probability between states $p(T)$, depending on whether the prerequisites were learned or not.

Learning Factors Analysis (LFA). In LFA [2], the probability p of a student performing correctly on an activity is modeled as a logistic function with parameters: n_i - number of times knowledge component i has been practiced, α - student learning parameter, β_i - coefficient of knowledge component i , γ - difficulty level of the activity

$$\ln\left(\frac{p}{1-p}\right) = \frac{\alpha}{|P_{kc}|} \sum_{i \in P_{kc}} n_i \beta_i - \gamma \quad (9)$$

where kc is the knowledge component associated with the activity and P_{kc} is the set containing kc along with its prerequisite knowledge components

Performance Factor Analysis (PFA). In PFA [11], the probability p of a student performing correctly on an activity is modeled as a logistic function with parameters: s_i - number of times knowledge component i has been used correctly, f_i - number of times knowledge component i has been used incorrectly, α - student learning parameter, β_i - coefficient for the success count of knowledge component i , η_i - coefficient for the failure count of knowledge component i , γ - difficulty level of the activity

$$\ln\left(\frac{p}{1-p}\right) = \frac{\alpha}{|P_{kc}|} \sum_{i \in P_{kc}} (s_i \beta_i + f_i \eta_i) - \gamma \quad (10)$$

Integrating Knowledge Tracing and Item Response Theory (KT-IRT). KT-IRT [7] combines the BKT model with item response theory. Here the student's knowledge is represented as a HMM with binary states. The probability of performing correctly on an activity based on knowledge component k at the i^{th} attempt is given by:

$$p_i = \begin{cases} p_g \left(\frac{1}{1+e^{-(\beta_k n_k + c_{k,0})}} \right) & \text{when } state_i = \text{not-learned} \\ (1 - p_s) \left(\frac{1}{1+e^{-(\beta_k n_k + c_{k,1})}} \right) & \text{when } state_i = \text{learned} \end{cases} \quad (11)$$

6.3 Parameters

Hyper-parameters for ZPDES: The best value of hyper-parameters such as η , β , γ and w_0 were found using grid search separately for each student model.

Change-Point Detection: For selecting the window size d and accuracy threshold t for ZPDES, and the threshold h for CUSUM, the FAR was fixed as $5e-05$. To find the FAR of the sliding window based method, a markov chain was constructed with the states corresponding to the number of ones to be obtained to reach the accuracy threshold t . With zero set as the absorbing state, the mean time to absorption for the chain was computed, which corresponds to $E_0(\tau)$. Similarly for CUSUM, a markov chain was constructed with the states corresponding to the possible values taken by the statistic $\tilde{U}_n$ and states with values greater than h were designated as the absorbing states. Again, the mean time to absorption was calculated to compute the FAR.

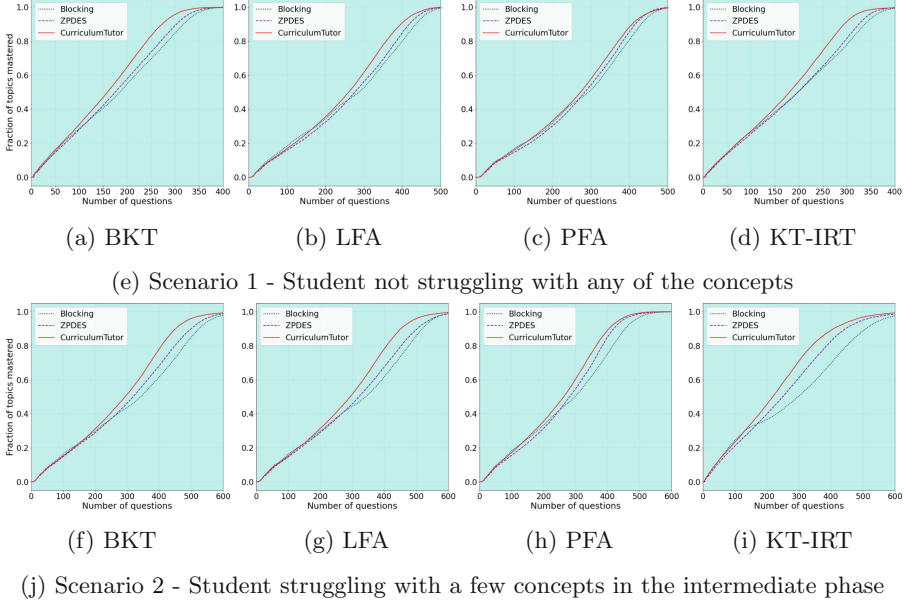
For the sliding window, the window size was set as 8 and the accuracy threshold as 0.7. Thus, out of the last 8 activities the student has to get at least 6 of them correct in order to infer that the concept has been mastered. Similarly the threshold h for CUSUM was set as $\ln(1/0.0004)$. For CUSUM p_{guess} and p_{slip} for each concept were sampled from $Beta(20, 40)$.

7 Results

In this section we present the experimental results that compare the performance of three tutoring algorithms namely *CurriculumTutor*, *ZPDES* and *Blocking* (a basic scheduling scheme where activities associated with the same concept are presented to a student until the concept is mastered, uses a sliding window to infer mastery). For these three algorithms, the following two scenarios were simulated using the four different student models:

Scenario 1. *Student not struggling with any of the concepts* Here similar model parameters (transition probabilities, learning coefficients, intercept values, etc.) were used for knowledge components (KC) of similar complexity and the parameter values were made smaller with increasing complexity. For instance, the KC parameters associated with two digit addition were lower as compared to those associated with one-digit addition.

Scenario 2. *Student struggling with a few concepts in the intermediate phase* This scenario was simulated by using a lower transition probability as well as learning parameters for the student model for one of the KCs - *three digit addition without carry and overflow*, as compared to Scenario 1. This causes the students to struggle on the concepts *AAACD* and *AAAACD* in Fig. 2.

**Fig. 3.** Simulation results

Learning gains were measured by computing the fraction of concepts in the curriculum graph that was mastered by the student over a given number of questions. The average curves for 500 runs were plotted for each student model, as seen in Fig. 3. It could be observed that for all the student models in both the scenarios, the learning curves of *CurriculumTutor* lie above those of ZPDES and Blocking, implying higher learning gains. In scenario 2, it could be observed that the learning curves of both the bandit algorithms, ZPDES and CurriculumTutor, are quite higher than that of Blocking (Fig. 3j). This is because while Blocking continuously tries to present activities related to a concept that the student is struggling with, the bandit algorithms performs exploration to find another concept that gives better rewards and thus contributes to higher learning progress. Thus **CurriculumTutor performs better than ZPDES and Blocking** in both the scenarios, irrespective of the student models chosen.

Limitations of the Approach: CurriculumTutor requires a curriculum graph that models the prerequisite relations between the concepts to be learned. When the curriculum graph has a chain-like structure, the algorithm cannot perform much better than Blocking as the scope for exploration becomes limited. Also the algorithm requires the user to define p_{guess} and p_{slip} for each concept.

8 Conclusion and Future Work

We have proposed *CurriculumTutor*, a novel multi-armed bandit and change-point detection based algorithm for adaptive activity sequencing to master a curriculum in the learning by doing setting. Simulation results show that CurriculumTutor significantly outperforms prior approaches in the benchmark domain of two operand addition up to a maximum of four digits. Future work on this problem would include accounting for forgetting of a learned concept, considering partial credit scores and accommodating for activities with varying levels of difficulty associated with a concept.

References

1. Andersen, E., Gulwani, S., Popovic, Z.: A trace-based framework for analyzing and synthesizing educational progressions. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 773–782 (2013)
2. Cen, H., Koedinger, K., Junker, B.: Learning factors analysis – a general method for cognitive model evaluation and improvement. In: Ikeda, M., Ashley, K.D., Chan, T.-W. (eds.) ITS 2006. LNCS, vol. 4053, pp. 164–175. Springer, Heidelberg (2006). https://doi.org/10.1007/11774303_17
3. Chaiklin, S.: The zone of proximal development in Vygotsky’s analysis of learning and instruction. In: Vygotsky’s Educational Theory in Cultural Context, vol. 1, pp. 39–64 (2003)
4. Clement, B., Roy, D., Oudeyer, P.Y., Lopes, M.: Multi-armed bandits for intelligent tutoring systems. arXiv preprint [arXiv:1310.3174](https://arxiv.org/abs/1310.3174) (2013)
5. Corbett, A.T., Anderson, J.R.: Knowledge tracing: modeling the acquisition of procedural knowledge. User Model. User-adapt. Interact. **4**(4), 253–278 (1994). <https://doi.org/10.1007/BF01099821>
6. Gordon, J., Zhu, L., Galstyan, A., Natarajan, P., Burns, G.: Modeling concept dependencies in a scientific corpus. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 866–875 (2016)
7. Khajah, M.M., Huang, Y., González-Brenes, J.P., Mozer, M.C., Brusilovsky, P.: Integrating knowledge tracing and item response theory: a tale of two frameworks. In: CEUR Workshop Proceedings, vol. 1181, pp. 7–15. University of Pittsburgh (2014)
8. Lattimore, T., Szepesvári, C.: Bandit Algorithms. Cambridge University Press, Cambridge (2020)
9. Lorden, G., et al.: Procedures for reacting to a change in distribution. Ann. Math. Stat. **42**(6), 1897–1908 (1971)
10. Mu, T., Goel, K., Brunskill, E.: Program2Tutor: combining automatic curriculum generation with multi-armed bandits for intelligent tutoring systems. In: Conference on Neural Information Processing Systems (2017)
11. Pavlik, P.I., Jr., Cen, H., Koedinger, K.R.: Performance factors analysis-a new alternative to knowledge tracing. Online Submission (2009)

12. Tartakovsky, A.G., Rozovskii, B.L., Blazek, R.B., Kim, H.: A novel approach to detection of intrusions in computer networks via adaptive sequential and batch-sequential change-point detection methods. *IEEE Trans. Signal Process.* **54**(9), 3372–3382 (2006)
13. Wang, S., et al.: Using prerequisites to extract concept maps from textbooks. In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, pp. 317–326 (2016)